Dynasties Phase One Framework v5

The Agentic Operating System for Global Freight Forwarding and Logistics

Prepared for: Dynasties.ai Founding Team

Date: March 13, 2026

Author: Manus AI

Classification: Internal — Architecture Specification and Engineering Blueprint

---

Executive Summary

Dynasties is an AI operating layer for the global freight forwarding and logistics industry. It sits above existing operational systems — CargoWise, Descartes, Magaya, Logitude, Softlink, BoxTop, and similar platforms — and converts unstructured operational inputs into structured operational actions using AI agents and deterministic execution services.

The foundational principle is absolute: **agents generate decisions; deterministic services execute them.** Language models never write directly to the system of record. Every AI output passes through validation, business rules, and human approval before it touches the database. This architecture allows Dynasties to integrate with existing systems immediately while evolving toward becoming the operational intelligence layer across global logistics.

This document is the complete Phase One architecture specification. It is structured so that it can be directly ingested by Replit Agent 4 for implementation. The framework covers fifteen sections: strategic positioning, system architecture, cargo insurance, risk intelligence, trade lane intelligence, exception handling, claims management, the Company Memory Graph, the Phase One operational workflow, the database model, the agent architecture, the core system rule, the design system for Replit Agent 4, the engineering build roadmap, and the output specification.

Phase One is not a minimum viable product. It is a durable architecture designed to remain technologically relevant for five years. It includes not only the core operational pipeline (email ingestion, document extraction, entity resolution, shipment construction, compliance screening, pricing, document generation, and invoicing) but also the strategic capabilities that define Dynasties as a new category: risk intelligence, trade lane intelligence, cargo insurance, exception handling, and claims management. These are not post-launch features. They are Phase One components.

The design system is optimized for Replit Agent 4 and Replit Design Mode. It uses React, Next.js, TailwindCSS, and the shadcn/ui component system with centralized design tokens. The UI architecture supports rapid design iteration on an infinite canvas, visual variant exploration, and theme-driven component consistency so that UI changes automatically propagate through production components. Design Mode prototypes convert directly into full application builds.

The strategic positioning draws from **Blue Ocean Strategy** [1] and **Peter Thiel's Zero to One** [2]. Dynasties does not compete with incumbent logistics platforms on features, integrations, or price. It creates a new category: an agentic logistics operating system where the platform performs operational tasks, agents orchestrate workflows, humans govern outcomes, and the system compounds intelligence from every shipment. The product experience is captured in one sentence: **"Forward your emails, documents, and legacy exports to Dynasties, and the platform automatically assembles a working operational system around your business."**

Section 1 — Strategic Positioning

Dynasties Defined

Dynasties is an AI operating layer for the global freight forwarding and logistics industry that sits above existing operational systems such as CargoWise, Descartes, Magaya, Logitude, Softlink, BoxTop, and similar platforms. The platform converts unstructured operational inputs — emails, documents, spreadsheets, XML files, EDI messages, and verbal instructions — into structured operational actions using AI agents and deterministic execution services.

The key principle that governs the entire architecture:

> **Agents generate decisions. Deterministic services execute them.**

This separation is what makes Dynasties production-grade rather than a prototype. Agents reason about data, classify commodities, assess risk, recommend carriers, and draft documents. Deterministic services create database records, generate PDFs, process financial transactions, bind insurance policies, and send communications. The two layers never cross. An agent never writes to the database. A service never makes a judgment call.

The Red Ocean

Global trade logistics software is a $15 billion market [3] dominated by platforms that all perform the same function: they provide a database with forms where humans enter

shipment data, generate documents from templates, and track cargo through manual milestone updates.

Platform	Category	What It Actually Is	Founded
CargoWise (WiseTech Global)	Enterprise TMS	Workflow database with deep customs integrations	2000
Descartes Systems	Logistics Network	Compliance database + logistics network	1981
Magaya	SMB TMS	Windows desktop TMS for small-to-mid forwarders	2001
Logitude	Cloud TMS	Web-based freight management for SMBs	2008
Softlink	Regional TMS	Freight forwarding software for Asia-Pacific	1986
BoxTop	Niche TMS	Freight management for specific verticals	2010
e2open	Supply Chain	Supply chain planning + visibility platform	2000
Flexport	Digital Forwarder	Digital forwarder with proprietary TMS	2013

These companies compete on the same dimensions: number of features, number of integrations, geographic coverage, and price per user. They are all workflow databases. They all require humans to perform the operational work. They all treat intelligence as an optional add-on rather than the core product. This is the red ocean — a crowded market where competition is fierce and differentiation is marginal.

The Blue Ocean

Dynasties creates a new market space by competing on dimensions that incumbents cannot match — because matching them would require rebuilding their products from scratch.

Red Ocean Dimension (Incumbents)	Blue Ocean Dimension (Dynasties)
Humans enter data into forms	Agents extract data from emails and documents
Static workflow templates	Adaptive workflows that learn from operator behavior
Intelligence as an add-on module	Intelligence as the core operating layer
Per-user pricing ($300-$450/user/month)	Per-shipment pricing (cost scales with value delivered)
Migration requires months of IT work	Migration requires forwarding emails
System stores what humans type	System performs the work humans used to do
Dashboard shows historical data	Workbench surfaces decisions that need governing
Insurance sold separately, manually	Insurance quoted automatically on every shipment
Risk assessed by human experience	Risk scored algorithmically per shipment
Exceptions handled reactively	Exceptions detected and resolved proactively

The Blue Ocean framework identifies four actions that create new market space [1]:

Eliminate. Manual data entry. Form-based interfaces. Per-user licensing. Multi-month implementation projects. Dedicated IT staff for system maintenance. Separate insurance quoting workflows. Manual exception triage.

Reduce. Operator time per shipment from 15-30 minutes to under 60 seconds. Number of screens to complete a workflow from 8-12 to a single approval view. Training time for new operators from weeks to hours. Dependence on any single operator's institutional knowledge. Time to resolve exceptions from days to minutes.

Raise. Speed of first value delivery from months to minutes. Data quality through AI extraction that is more consistent than human data entry. Compliance coverage to 100% of shipments screened, not the 30-40% that get manual review. Insurance attachment rate from 10-15% to 30%+ through embedded quoting. Operational visibility through logged, reasoned, and auditable agent actions.

Create. The Company Memory Graph — a persistent intelligence layer that learns how each company operates. The Agentic Workbench — an interface where operators govern AI-driven operations rather than performing manual tasks. Risk Intelligence — algorithmic risk

scoring on every shipment. Trade Lane Intelligence — predictive analytics for cost, delay, and documentation risk. Automated Exception Handling — proactive detection and resolution of operational disruptions. Embedded Insurance — cargo insurance quoted and bound within the shipment workflow. Claims Management — AI-assisted claim preparation and submission.

The Zero-to-One Advantage

Peter Thiel's framework asks: "What important truth do very few people agree with you on?" [2] The Dynasties answer: **freight forwarding operations can be performed by AI agents, not just assisted by software tools.** Every incumbent believes that logistics software should help humans work faster. Dynasties believes that logistics software should do the work, with humans governing the outcomes.

This is a 0 to 1 leap because it changes the unit economics of freight forwarding. Today, a forwarder's cost structure is dominated by labor — operators who enter data, generate documents, check compliance, calculate charges, assess risk, arrange insurance, and send invoices. Dynasties collapses that labor cost by 80-90%, enabling a forwarder to handle 10x the shipment volume with the same team. This is not an incremental improvement. It is a structural transformation of the business model.

The technological moat compounds over time through five reinforcing mechanisms: the Company Memory Graph accumulates operational intelligence that cannot be replicated. The cross-client intelligence network creates data advantages that grow with each new customer. The learning loops improve agent accuracy with every operator correction. The insurance and financial layers create independent revenue streams that increase switching costs. The risk intelligence layer becomes more accurate as it processes more shipments across more trade lanes. Incumbents cannot replicate these advantages by adding AI features to their existing platforms — their architectures are built around human data entry, and retrofitting agent-native intelligence into legacy codebases is a multi-year rewrite that would alienate their existing customer base.

The conceptual analogies are deliberate: Dynasties aims to be **AWS for logistics operations** (infrastructure that other businesses build on) and **Stripe for global trade workflows** (complex processes reduced to simple API calls with intelligence embedded in every transaction).

Section 2 — System Architecture

Architecture Overview

The Dynasties platform is organized into three service tiers: Operational Services that handle the core freight forwarding workflow, Financial and Intelligence Services that provide strategic capabilities, and Infrastructure Services that support the platform. All services communicate through asynchronous message queues (SQS) and event topics (SNS). No service calls another service directly via HTTP in the critical path. This ensures that a slow or failing service does not cascade failures across the platform.

Operational Services

These services handle the daily freight forwarding workflow — the operations that a forwarder cannot function without.

Service	Responsibility	Key Operations
Email Ingestion Service	Receives inbound emails from SES, parses MIME content, extracts attachments, classifies document types, and publishes ingestion tasks to the agent queue.	MIME parsing, attachment extraction, document type identification, S3 storage of raw emails and attachments
Document Extraction Service	Processes documents (PDF, DOCX, XLSX, images) through OCR and AI extraction to produce structured data. Handles BOLs, commercial invoices, packing lists, certificates of origin, customs declarations, and arrival notices.	OCR via AWS Textract, AI extraction via Claude Sonnet 4.6, structured JSON output with field-level confidence scores
Entity Resolution Service	Resolves party names and addresses from extracted data against the Company Memory Graph. Handles fuzzy matching, deduplication, address normalization, and auto-creation of new entity records.	Entity matching, deduplication, address standardization, new entity creation with approval
Shipment Construction Service	Assembles complete shipment records from extracted and resolved data. Creates shipment drafts with all fields populated, linked to entities, commodities, and	Shipment record creation, container management, milestone tracking, status transitions

	containers. Manages shipment lifecycle states.	
Compliance Screening Service	Screens all parties and commodities against sanctions lists (SDN, Entity List, Denied Persons, UN/EU lists), export control regulations, and restricted commodity databases. This service is mandatory and non-bypassable — every shipment passes through compliance before approval.	Restricted party screening, export control verification, denied party checks, screening result storage with match details
Pricing and Charge Calculation Service	Calculates charges for shipments based on rate tables, historical pricing patterns, and Company Memory Graph preferences. Supports freight charges, handling fees, customs brokerage, documentation fees, and ad-hoc charges. Performs margin analysis and anomaly detection.	Rate table management, charge calculation, margin analysis, anomaly flagging, charge approval workflow
Document Generation Service	Generates operational documents from approved shipment records: Bills of Lading (ocean and house), Air Waybills, Cargo Manifests, Arrival Notices, Packing Lists, and Certificates of Origin. Uses template engine with AI-assisted intelligent filling.	Template rendering, PDF generation, document versioning, S3 storage, email delivery
Billing and Invoicing Service	Generates invoices from approved charges, produces PDF output, delivers via email (SES), and tracks accounts receivable with aging. Supports credit notes, payment recording, and statement generation.	Invoice generation, PDF rendering, email delivery, AR tracking, payment reconciliation

Financial and Intelligence Services

These services provide the strategic capabilities that differentiate Dynasties from every incumbent platform. They are Phase One components, not post-launch features.

Service	Responsibility	Key Operations
Insurance Service	Manages cargo insurance quoting, insurer API integrations, coverage recommendations, policy binding, certificate of insurance generation, and policy attachment to shipment documents. Integrates with Loadsure, Breeze, and other cargo insurance providers via their APIs.	Quote generation, policy binding, certificate issuance, premium tracking, insurer API management
Risk Intelligence Service	Calculates an operational risk score for each shipment based on trade lane risk, carrier reliability, commodity risk classification, historical delay patterns, port congestion data, geopolitical instability indicators, and weather risk. The risk score feeds into insurance quoting, compliance prioritization, and pricing decisions.	Risk score calculation, factor weighting, mitigation recommendation, historical risk tracking
Trade Lane Intelligence Service	Learns from historical shipment data across the platform to provide predictive analytics: cost prediction, delay prediction, documentation error prediction, and customs risk prediction per trade lane. Improves with every shipment processed.	Cost forecasting, transit time prediction, risk alerting, routing recommendation
Claims Management Service	Manages the cargo claims workflow: incident documentation, insurance claim preparation, insurer	Incident recording, evidence collection, claim package assembly, insurer

	communication logging, evidence storage, and claim status tracking. Prepares structured claim packages ready for submission to insurers.	communication, status tracking
Exception Management Service	Detects and manages operational exceptions: missing documentation, customs holds, carrier changes, shipment delays, regulatory flags, and billing discrepancies. Proposes resolutions and prepares operator actions for rapid triage.	Exception detection, resolution proposal, operator action preparation, escalation management

Infrastructure Services

Service	Responsibility
Workflow Orchestrator	Deterministic workflow engine (LangGraph) and rules engine. Manages state transitions, approval gates, parallel branches, retry logic, and escalation paths. Separate from Agent Worker to ensure workflow logic is never blocked by LLM latency.
Model Routing Service	All LLM API connections, model selection, fallback chains, response caching, confidence scoring, prompt version management, and cost tracking. No other service calls LLM APIs directly.
Agent Worker Service	Runs LangGraph agent orchestration. Consumes tasks from SQS, invokes LLM APIs via Model Router, produces structured JSON outputs. Hosts all specialist agents.
API Gateway Service	All HTTP requests from the frontend. Authentication, authorization, request validation, response formatting. REST API for web app and future integrations.

Service Communication

Services communicate through two mechanisms. **SQS queues** handle task distribution — when the Email Ingestion Service receives a new email, it publishes a task to the `intake-tasks` queue, which the Agent Worker Service consumes. Each agent type has its own queue: `intake-tasks`, `classification-tasks`, `compliance-tasks`, `risk-tasks`, `insurance-tasks`, `document-tasks`, `billing-tasks`, `exception-tasks`, `claims-tasks`, `migration-tasks`. Dead-letter queues capture failures for retry and debugging.

SNS topics handle event broadcasting — when a shipment is created, the Shipment Construction Service publishes to the `shipment-created` topic, which triggers downstream workflows in the Risk Intelligence Service, Insurance Service, and Document Generation Service simultaneously. Event topics include: `shipment-created`, `shipment-updated`, `shipment-approved`, `compliance-alert`, `risk-score-generated`, `insurance-quote-generated`, `document-generated`, `invoice-created`, `exception-detected`, `claim-filed`.

Section 3 — Cargo Insurance Architecture

Insurance Service

The Insurance Service manages the complete cargo insurance lifecycle within the Dynasties platform. Insurance is not a bolt-on feature — it is embedded in the shipment workflow. Every shipment that passes through Dynasties receives an automatic insurance quote as part of the standard processing pipeline, before the operator reaches the approval screen.

Responsibilities:

The Insurance Service handles cargo insurance quoting by calling insurer APIs with shipment parameters, coverage recommendation based on cargo value, commodity risk, and trade lane history, policy binding when the operator approves coverage, certificate of insurance generation as a PDF document attached to the shipment, policy attachment to shipment documents so that the insurance certificate travels with the BOL and commercial invoice, and premium tracking for accounting and commission reconciliation.

Supported Integration Targets:

Insurer Platform	Integration Type	Coverage Types	Quote Speed
Loadsure	REST API	All-risk marine cargo, warehouse-to-warehouse	Real-time (< 2 seconds)

Breeze AI	REST API	Per-shipment cargo, open cargo, contingent cargo	Real-time (< 2 seconds)
Additional providers	REST API (standardized adapter pattern)	Varies by provider	Provider-dependent

The Insurance Service uses an adapter pattern for insurer integrations. Each insurer has a dedicated adapter that translates Dynasties' standardized insurance request format into the insurer's specific API format and normalizes the response. Adding a new insurer requires writing one adapter — no changes to the core service or the Cargo Insurance Agent.

Cargo Insurance Agent

The Cargo Insurance Agent is a specialist agent in the Agent Worker Service that generates insurance recommendations. It does not bind policies or issue certificates — those are deterministic actions performed by the Insurance Service after operator approval.

Inputs (structured JSON):

The agent receives the complete shipment context: shipment data (origin, destination, mode, carrier, transit time), cargo value (from commercial invoice or operator entry), commodity type and HS code (from Classification Agent), trade lane (from Trade Lane Intelligence Service), carrier reliability score (from Risk Intelligence Service), and the overall risk score for the shipment.

Outputs (structured JSON):

The agent produces a coverage recommendation (all-risk, named perils, or warehouse-to-warehouse with reasoning), a premium estimate range based on historical quotes for similar shipments, a recommended insurer (selected from available providers based on coverage fit, price, and claims history), and a confidence score indicating the reliability of the recommendation.

Execution Flow:

The Cargo Insurance Agent produces its structured JSON output. The output is presented to the operator in the Agentic Workbench alongside the shipment draft. If the operator approves insurance coverage, the Insurance Service calls the selected insurer's API to bind the policy, generates the certificate of insurance, attaches it to the shipment documents, and records the premium for accounting. The agent never directly calls an insurer API. The agent never writes to the database. The Insurance Service executes all actions.

Revenue Model

Dynasties earns revenue from insurance in two ways. As a technology platform without an insurance license, Dynasties receives a revenue share of 10-15% of the premium from the insurer partner on every policy bound through the platform. With an insurance agent license, Dynasties receives a commission of 15-25% of the premium as the agent of record. The embedded insurance model increases the forwarder's insurance attachment rate from the industry average of 10-15% (manual quoting) to 30%+ (automated quoting at point of booking), creating significant incremental revenue for both the forwarder and Dynasties.

Section 4 — Risk Intelligence

Risk Intelligence Service

The Risk Intelligence Service calculates an operational risk score for each shipment. This score is not a standalone metric — it feeds directly into three other services: the Insurance Service (risk score affects premium calculation and coverage recommendation), the Compliance Screening Service (high-risk shipments receive enhanced screening), and the Pricing and Charge Calculation Service (risk-adjusted pricing for high-risk shipments).

Risk Factors:

Factor	Data Source	Weight	Update Frequency
Trade lane risk	Historical shipment data, port congestion indices, customs clearance times	High	Daily
Carrier reliability	On-time delivery rates, damage claim history, equipment availability	High	Weekly
Commodity risk	HS code risk classification, hazmat indicators, perishability, value density	Medium	Static (updated with tariff changes)
Historical delays	Company Memory Graph — past delays on this lane, with this carrier, for this commodity	Medium	Per shipment

Port congestion	Real-time port congestion data, vessel queue lengths, terminal utilization	High	Hourly
Geopolitical instability	Country risk indices, trade restriction changes, sanctions updates	Medium	Daily
Weather risk	Seasonal weather patterns, active weather events along route	Low-Medium	Daily

Risk Score Calculation:

The Risk Intelligence Service produces a composite risk score on a 0-100 scale (0 = minimal risk, 100 = extreme risk) with a breakdown by factor. The score is deterministic — given the same inputs, it produces the same score. The factor weights are configurable per company via the Company Memory Graph (a forwarder specializing in perishable goods may weight weather risk higher than a general cargo forwarder).

Risk Intelligence Agent

The Risk Intelligence Agent interprets the raw risk score and produces actionable intelligence for the operator.

Outputs (structured JSON):

The agent produces a risk score (the composite 0-100 value), an explanation in natural language describing the primary risk drivers for this specific shipment, and recommended mitigation actions — concrete steps the operator can take to reduce risk (alternative routing, carrier change, additional insurance coverage, pre-clearance documentation, weather delay buffer in transit time estimates).

Cross-Service Integration:

When the Risk Intelligence Service generates a score, it publishes to the `risk-score-generated` SNS topic. The Insurance Service subscribes and uses the risk score to adjust the coverage recommendation and premium estimate. The Compliance Screening Service subscribes and escalates screening depth for shipments above a configurable risk threshold. The Pricing and Charge Calculation Service subscribes and can apply risk-adjusted surcharges or flag shipments where the margin does not adequately compensate for the risk level.

Section 5 — Trade Lane Intelligence

Trade Lane Intelligence Service

The Trade Lane Intelligence Service learns from historical shipment data across the platform to provide predictive analytics for every trade lane. Unlike the Risk Intelligence Service (which scores individual shipments), the Trade Lane Intelligence Service builds aggregate intelligence about routes — patterns that emerge from hundreds or thousands of shipments on the same origin-destination-mode combination.

Capabilities:

Capability	What It Predicts	Accuracy Improves With
Cost prediction	Expected total landed cost for a shipment on a given lane, including freight, handling, customs, and documentation fees	More shipments on the same lane, more carrier rate data
Delay prediction	Probability and expected duration of delays based on historical patterns, seasonal trends, and current conditions	More shipments, more milestone data, more exception records
Documentation error prediction	Likelihood of documentation issues (missing fields, incorrect classifications, formatting errors) based on lane-specific customs requirements	More shipments, more customs feedback, more operator corrections
Customs risk prediction	Probability of customs holds, inspections, or duty reassessments based on commodity-lane combinations and historical customs outcomes	More shipments, more customs outcome data

Data Sources:

The Trade Lane Intelligence Service draws from three data pools. The first is the company-specific data in the Company Memory Graph — the forwarder's own shipment history on each lane. The second is the cross-client anonymized data from all Dynasties customers operating on the same lane (with appropriate consent and anonymization). The third is

external data feeds: port congestion indices, carrier schedule reliability reports, and customs processing time statistics.

Trade Lane Intelligence Agent

The Trade Lane Intelligence Agent produces predictive outputs for each shipment based on its trade lane.

Outputs (structured JSON):

The agent produces a predicted cost range (low, expected, high) with confidence intervals, an expected transit time with probability distribution, risk alerts specific to the trade lane (current congestion, recent customs holds, seasonal patterns), and recommended routing alternatives if the primary route shows elevated risk or cost.

Learning Mechanism:

Every completed shipment on a trade lane updates the intelligence model. The actual cost is compared to the predicted cost. The actual transit time is compared to the predicted transit time. Documentation issues and customs outcomes are recorded. Over time, the predictions become increasingly accurate — the system learns the true cost and timing characteristics of each lane from real operational data, not from carrier marketing materials or industry averages.

Section 6 — Exception Handling

Exception Management Service

Freight forwarding is defined by exceptions. A shipment that moves from origin to destination without any deviation is the ideal — but it is not the norm. Containers are rolled to the next vessel. Documents arrive incomplete. Customs holds delay clearance. Carriers change schedules. Regulatory requirements shift. In traditional systems, operators discover these exceptions through manual monitoring, phone calls, and email chains. By the time the exception is identified, the cost of resolution has already escalated.

The Exception Management Service detects exceptions proactively and proposes resolutions before the operator is aware of the problem.

Exception Categories:

Category	Detection Method	Example
Missing documentation	Comparison of required document checklist (per trade lane, per commodity,	Shipment to Brazil requires a Certificate of Origin; none is attached

	per customs authority) against documents attached to the shipment	
Customs hold detection	Milestone monitoring — if customs clearance exceeds the expected timeframe for the lane, flag as potential hold	Shipment cleared customs in 2 hours on last 10 shipments; current shipment has been in customs for 18 hours
Carrier changes	Carrier notification parsing (email or API), vessel schedule monitoring	Carrier notifies that container has been rolled to next sailing
Shipment delays	ETA comparison against predicted transit time from Trade Lane Intelligence	Vessel ETA revised from March 20 to March 25 — 5-day delay
Regulatory flags	Real-time monitoring of trade regulation changes against active shipments	New tariff imposed on commodity class affecting 3 active shipments
Billing discrepancies	Comparison of vendor invoices against expected charges from rate tables and historical patterns	Carrier invoice shows $2,400 for ocean freight; rate table and historical average is $1,800

Operations Exception Agent

The Operations Exception Agent does not just detect problems — it proposes solutions. When an exception is detected, the agent analyzes the shipment context, the Company Memory Graph (how has this company handled similar exceptions before?), and the available options, then produces a structured resolution proposal.

Outputs (structured JSON):

The agent produces an exception classification (category, severity, affected shipment), an impact assessment (cost impact, time impact, compliance impact), proposed resolution actions (specific steps the operator can take, ranked by effectiveness), required communications (draft emails or notifications to affected parties), and an escalation recommendation if the exception exceeds the agent's confidence threshold.

Operator Interaction:

Exceptions appear in the Agentic Workbench as a dedicated queue, sorted by severity. Each exception shows the detection reason, the proposed resolution, and one-click action buttons. The operator can approve the proposed resolution (the Exception Management

Service executes it), modify the resolution (the correction trains the Memory Agent), or escalate to a manager. The goal is resolution in minutes, not days.

Section 7 — Claims Management

Claims Service

When cargo is damaged, lost, or delayed beyond contractual terms, the forwarder must file an insurance claim. This process is traditionally manual, document-intensive, and slow — forwarders gather evidence from multiple sources, compile claim packages in Word documents, and exchange emails with insurers over weeks or months. The Claims Service automates this workflow.

Capabilities:

The Claims Service handles incident documentation (structured recording of what happened, when, where, and the extent of damage or loss), insurance claim preparation (assembling all required documents, photographs, and evidence into a structured claim package that meets insurer requirements), insurer communication logging (tracking all correspondence with the insurer, including submissions, requests for additional information, and settlement offers), evidence storage (centralized, versioned storage of all claim-related documents, photographs, and correspondence in S3), and claim status tracking (lifecycle management from incident through submission, review, negotiation, and settlement).

Claims Preparation Agent

The Claims Preparation Agent accelerates claim preparation from days to minutes by automatically assembling the claim package from data already in the system.

Inputs (structured JSON):

The agent receives the shipment record (complete shipment data including all parties, commodities, containers, and milestones), the insurance policy (coverage type, policy number, insurer, coverage limits, deductible), damage reports (structured incident data entered by the operator or parsed from damage survey reports), and supporting documents (BOL, commercial invoice, packing list, photographs, survey reports — all already stored in S3 from the shipment workflow).

Outputs (structured JSON):

The agent produces a structured claim package ready for submission, including a claim narrative (AI-generated description of the incident meeting insurer requirements), a document manifest (list of all supporting documents with S3 references), a loss calculation

(cargo value, salvage value, deductible, net claim amount), and an insurer-specific submission format (adapted to the requirements of the specific insurer via the Insurance Service adapter pattern).

Execution Flow:

The operator initiates a claim from the shipment detail view in the Workbench. The Claims Preparation Agent assembles the package automatically — most of the required information is already in the system from the original shipment workflow. The operator reviews the claim package, adds any additional evidence (photographs, survey reports), and approves submission. The Claims Service submits the claim to the insurer via API (if supported) or generates a formatted PDF package for email submission. All subsequent insurer communications are logged and tracked in the Claims Service.

Section 8 — Company Memory Graph

What It Is

The Company Memory Graph is a persistent, evolving knowledge structure that represents everything Dynasties has learned about a company's operations. It is not a static database of records. It is a living intelligence layer that grows with every shipment, every email, every operator correction, and every document processed. It is the primary long-term strategic moat of the platform.

Traditional TMS platforms store records — shipments, invoices, entities. The Company Memory Graph stores **relationships, patterns, preferences, and precedents.** It knows not just that Company X shipped electronic components from Shenzhen to Miami, but that Company X always uses Maersk for this lane, always requires a certificate of origin, always marks the shipment as "fragile," always applies a 15% markup on freight charges, always sends the arrival notice to three specific email addresses at the consignee, and always purchases all-risk insurance for shipments above $50,000 in cargo value.

Graph Structure

The Company Memory Graph stores relationships between the following entity types, with edges representing operational patterns and preferences.

Entity Type	What It Represents	Key Relationships
Companies	The forwarder's own company profile, including operational preferences,	Parent of all other entities

	approval policies, and default settings	
Customers	Shippers, importers, exporters — the forwarder's clients	Linked to consignees, commodities, trade lanes, carriers, insurance policies
Consignees	Receiving parties at destination	Linked to customers, trade lanes, documentation requirements
Carriers	Shipping lines, airlines, trucking companies	Linked to trade lanes, reliability scores, rate history, claims history
Trade Lanes	Origin-destination-mode combinations	Linked to carriers, transit times, cost patterns, risk scores, customs requirements
Commodities	Product categories with HS codes	Linked to classification precedents, compliance requirements, insurance risk profiles
Documents	Document types and templates	Linked to customers, trade lanes, regulatory requirements
Shipments	Historical shipment records	Linked to all other entities — the primary data source for pattern learning
Insurance Policies	Coverage records with premium and claims history	Linked to shipments, carriers, commodities, trade lanes, claims

How Agents Use It

Every agent in the system queries the Company Memory Graph before producing its output. This is what makes Dynasties fundamentally different from a generic AI tool — the agents do not operate from general knowledge alone. They operate from the specific, accumulated intelligence of each company's operations.

The Intake Agent queries: "What are this customer's standard shipping patterns? What documents do they typically attach? What fields do they usually include or omit?" The Classification Agent queries: "Has this company shipped this commodity before? What HS code was used? Was it corrected?" The Carrier Agent queries: "Does this customer have a

preferred carrier for this lane? What transit times have they accepted historically?" The Insurance Agent queries: "Does this customer typically purchase insurance? What coverage level? What is their claims history on this lane?" The Risk Intelligence Agent queries: "What exceptions has this company experienced on this lane? What was the resolution?"

Learning Loops

Every operator correction improves the system. This is an engineered feedback mechanism with specific data flows, not a vague aspiration.

Correction Type	System Response	Impact Timeline
HS code changed	Stores precedent in Memory Graph. After 3 identical corrections, promotes to deterministic rule in Rules Engine.	Immediate for same commodity; rule after 3 occurrences
Carrier changed	Records preference in customer profile. Adjusts Carrier Agent weighting for this customer-lane combination.	Next shipment on same lane
Charge adjusted	Updates pricing pattern. Flags future charges that deviate from the corrected pattern.	Next similar shipment
Entity corrected	Updates entity record. Improves Entity Agent fuzzy matching for similar names/addresses.	Immediate
Document modified	Records template preference. Document Agent uses corrected format for future documents of same type for this customer.	Next document generation
Insurance declined	Records preference. Reduces insurance recommendation frequency for this customer-commodity-lane combination.	Next similar shipment
Insurance coverage changed	Records preferred coverage level. Insurance Agent adjusts future recommendations.	Next similar shipment

Exception resolution modified	Records preferred resolution approach. Exception Agent proposes similar resolutions for future exceptions of same type.	Next similar exception
Risk mitigation overridden	Records risk tolerance. Risk Intelligence Agent adjusts factor weighting for this company.	Next risk assessment

The **auto-approval rate** is the single most important KPI for the platform. A new customer on Day 1 might have a 60% auto-approval rate. After 3 months, it should reach 85%. After 12 months, 95%+. The remaining 5% represents genuinely novel situations that require human judgment — and those situations train the system for the next novel case.

The Strategic Moat

The Company Memory Graph creates three compounding advantages. **Switching cost inversion:** traditional TMS platforms have low switching costs because the data is tabular and exportable. The Memory Graph creates high switching costs because operational intelligence — patterns, preferences, precedents, and learned behaviors — cannot be exported as a CSV. **Increasing autonomy:** as the graph accumulates more data, the rules engine handles more cases deterministically, agents produce higher-confidence outputs, and approval thresholds can be raised. The platform gets cheaper to operate and more valuable to the customer simultaneously. **Cross-client intelligence:** with appropriate anonymization and consent, patterns learned from one company's operations benefit all companies on the platform, creating a network effect where each new customer makes the intelligence better for every existing customer.

Section 9 — Phase One Operational Workflow

The Complete Email-to-Shipment Flow

This section documents the end-to-end operational workflow for the most common Phase One use case: an operator receives a booking request via email, and Dynasties processes it into a fully documented, insured, and invoiced shipment. This workflow is the reference implementation that validates the entire architecture.

Step 1 — Email Receipt. An email arrives at `ops@dynasties.ai` (or a company-specific address like `intake@acmefreight.dynasties.ai`). AWS SES receives the email and triggers a

Lambda function that stores the raw email in S3 and publishes a message to the `email-intake` SQS queue.

Step 2 — Email Parsing. The Email Ingestion Service consumes the message, parses the MIME content, extracts the body text, identifies and extracts all attachments (PDFs, images, spreadsheets), classifies each attachment by document type (commercial invoice, packing list, BOL, booking confirmation, rate quote), and stores each attachment in S3 with metadata. It publishes a structured intake task to the `agent-intake` queue.

Step 3 — Document Extraction. The Agent Worker Service consumes the intake task and routes it to the Document Extraction pipeline. Each document is processed by Claude Sonnet 4.6 (via the Model Routing Service) to extract structured data: party names and addresses, commodity descriptions, quantities, weights, dimensions, values, HS codes, container numbers, vessel names, port codes, and dates. Each extracted field includes a confidence score. The extraction results are published as structured JSON.

Step 4 — Entity Resolution. The Entity Resolution Service receives the extracted party data and resolves each party against the Company Memory Graph. It performs fuzzy matching on names and addresses, deduplicates against existing records, normalizes addresses, and either links to an existing entity or creates a new entity draft for operator confirmation. The resolved entities are attached to the shipment draft.

Step 5 — Shipment Draft Construction. The Shipment Construction Service assembles a complete shipment record from the extracted and resolved data. It populates all standard fields: origin, destination, mode (ocean, air, ground), service type (FCL, LCL, FTL, LTL), incoterms, commodity details, container specifications, parties (shipper, consignee, notify party, carrier, agent), and milestone dates. Fields that could not be extracted with sufficient confidence are flagged as "needs review."

Step 6 — Parallel Intelligence Processing. Three agents process the shipment draft simultaneously. The Classification Agent assigns HS codes with confidence scores, checking the Company Memory Graph for precedents. The Compliance Screening Service screens all parties against sanctions lists and export controls. The Risk Intelligence Service calculates the composite risk score. These three processes run in parallel because they are independent — none depends on the output of the others.

Step 7 — Insurance Quoting. After the risk score is generated, the Cargo Insurance Agent produces a coverage recommendation and the Insurance Service calls the insurer API to generate a real-time quote. The quote, coverage recommendation, and premium are attached to the shipment draft.

Step 8 — Trade Lane Intelligence. The Trade Lane Intelligence Service provides predictive analytics for the shipment's trade lane: expected cost range, expected transit time, documentation risk alerts, and customs risk assessment. These predictions are attached to the shipment draft for operator reference.

Step 9 — Pricing. The Pricing and Charge Calculation Service calculates all charges: freight, handling, documentation, customs brokerage, and any applicable surcharges. It checks rate tables, historical pricing patterns from the Company Memory Graph, and flags any anomalies. The charge breakdown is attached to the shipment draft.

Step 10 — Workbench Presentation. The complete shipment draft — with all extracted data, resolved entities, HS codes, compliance results, risk score, insurance quote, trade lane predictions, and calculated charges — appears in the operator's Agentic Workbench. Every AI-generated field shows its confidence score and reasoning. Fields below the confidence threshold are highlighted for review.

Step 11 — Operator Approval. The operator reviews the draft. They can approve the entire shipment with one click, modify specific fields (each modification is captured as a learning event for the Memory Agent), approve or decline insurance coverage, adjust charges, and add notes. The approval triggers the execution phase.

Step 12 — Execution. Upon approval, deterministic services execute in sequence: the Shipment Construction Service finalizes the shipment record in the database, the Document Generation Service produces the BOL, AWB, or other required documents, the Insurance Service binds the policy (if approved) and generates the certificate of insurance, the Billing and Invoicing Service generates the invoice and delivers it via email, and the shipment enters active tracking.

Total elapsed time from email receipt to approved shipment: 60-90 seconds of operator time. The system processing time (Steps 1-9) is approximately 15-30 seconds. The operator review time (Steps 10-11) depends on the complexity of the shipment and the auto-approval rate, but for a well-trained system with a 90%+ auto-approval rate, most shipments require only a glance and a click.

Section 10 — Database Model

Schema Design

The database uses PostgreSQL with the pgvector extension for embedding storage (used by the Company Memory Graph for semantic similarity searches). The schema is designed around the operational entities of freight forwarding, with additional tables for the intelligence and financial layers.

Core Operational Tables

Table	Purpose	Key Fields

`companies`	Forwarder company profiles	`id`, `name`, `address`, `settings_json`, `subscription_tier`, `created_at`
`users`	Operator accounts within a company	`id`, `company_id`, `email`, `name`, `role` (admin, manager, operator, viewer), `preferences_json`
`entities`	All parties: customers, consignees, carriers, vendors, agents	`id`, `company_id`, `entity_type`, `name`, `address_json`, `contact_json`, `tax_id`, `aliases_json`, `created_by` (manual, ai_extracted, imported)
`shipments`	Core shipment records	`id`, `company_id`, `reference_number`, `mode` (ocean, air, ground), `service_type` (FCL, LCL, FTL, LTL), `direction` (import, export, cross-trade), `status`, `origin_json`, `destination_json`, `incoterms`, `shipper_entity_id`, `consignee_entity_id`, `notify_entity_id`, `carrier_entity_id`, `agent_entity_id`, `commodity_json`, `container_json`, `milestones_json`, `risk_score`, `created_at`, `approved_at`, `approved_by`
`shipment_documents`	Documents attached to shipments	`id`, `shipment_id`, `document_type` (BOL, AWB, commercial_invoice, packing_list, COO, arrival_notice, COI), `s3_key`, `version`, `generated_by` (agent, manual, imported), `created_at`
`shipment_charges`	Individual charge line items	`id`, `shipment_id`, `charge_type` (freight, handling, documentation, customs, insurance_premium, surcharge), `description`,

		`amount`, `currency`, `rate_source` (rate_table, historical, manual), `approved`, `approved_by`
`invoices`	Generated invoices	`id`, `company_id`, `shipment_id`, `invoice_number`, `customer_entity_id`, `total_amount`, `currency`, `status` (draft, sent, paid, overdue), `due_date`, `s3_key`, `sent_at`, `paid_at`
`containers`	Container records linked to shipments	`id`, `shipment_id`, `container_number`, `container_type` (20GP, 40GP, 40HC, 45HC, 20RF, 40RF), `seal_number`, `weight_kg`, `volume_cbm`, `commodity_description`

Intelligence Tables

Table	Purpose	Key Fields
`memory_graph_nodes`	Nodes in the Company Memory Graph	`id`, `company_id`, `node_type` (entity, commodity, trade_lane, carrier, document_template, pricing_pattern), `data_json`, `embedding` (vector), `created_at`, `updated_at`
`memory_graph_edges`	Relationships between graph nodes	`id`, `source_node_id`, `target_node_id`, `edge_type` (prefers, ships_via, classifies_as, prices_at, requires_document, insures_with), `weight`, `metadata_json`, `created_at`
`classification_precedents`	HS code classification history	`id`, `company_id`, `commodity_description`, `hs_code`, `confidence`, `source` (agent, operator_correction,

		imported), `shipment_id`, `created_at`
`compliance_screenings`	Screening results for audit trail	`id`, `shipment_id`, `screened_entity_id`, `screening_type` (SDN, entity_list, denied_persons, UN, EU), `result` (clear, potential_match, confirmed_match), `match_details_json`, `screened_at`
`risk_scores`	Historical risk scores per shipment	`id`, `shipment_id`, `composite_score`, `factor_breakdown_json`, `mitigation_recommendations_jso n`, `calculated_at`
`trade_lane_stats`	Aggregate trade lane intelligence	`id`, `company_id`, `origin_code`, `destination_code`, `mode`, `avg_cost`, `avg_transit_days`, `delay_probability`, `documentation_error_rate`, `customs_hold_rate`, `sample_size`, `last_updated`
`operator_corrections`	Learning events from operator modifications	`id`, `company_id`, `user_id`, `shipment_id`, `agent_type`, `field_name`, `original_value`, `corrected_value`, `scope` (this_shipment, this_customer, global), `created_at`
`rules`	Deterministic rules generated from patterns	`id`, `company_id`, `rule_type`, `condition_json`, `action_json`, `source` (manual, learned), `confidence`, `activation_count`, `created_at`

Financial and Insurance Tables

Table	Purpose	Key Fields

insurance_quotes	Insurance quotes generated per shipment	id , shipment_id , insurer , coverage_type , cargo_value , premium , deductible , quote_reference , valid_until , status (quoted, accepted, declined, expired), created_at
insurance_policies	Bound insurance policies	id , shipment_id , quote_id , insurer , policy_number , coverage_type , cargo_value , premium , deductible , effective_date , expiry_date , certificate_s3_key , bound_at
claims	Insurance claims	id , company_id , shipment_id , policy_id , claim_type (damage, loss, delay, shortage), incident_date , incident_description , claimed_amount , settlement_amount , status (draft, submitted, under_review, additional_info_requested, settled, denied), claim_package_s3_key , created_at , settled_at
claim_communications	Insurer correspondence log	id , claim_id , direction (inbound, outbound), communication_type (email, document, note), subject , content , attachments_json , created_at
exceptions	Operational exceptions	id , shipment_id , exception_type (missing_document, customs_hold, carrier_change, delay, regulatory_flag, billing_discrepancy), severity (low, medium, high, critical), description , proposed_resolution_json , actual_resolution_json , status

		(detected, acknowledged, in_progress, resolved, escalated), `detected_at`, `resolved_at`, `resolved_by`
`rate_tables`	Carrier and service rate tables	`id`, `company_id`, `carrier_entity_id`, `origin_code`, `destination_code`, `mode`, `service_type`, `rate_json`, `effective_date`, `expiry_date`, `created_at`

Email and Ingestion Tables

Table	Purpose	Key Fields
`ingested_emails`	Raw email records	`id`, `company_id`, `from_address`, `to_address`, `subject`, `received_at`, `s3_key_raw`, `processing_status` (received, parsed, extracted, processed, failed), `shipment_id` (linked after processing)
`ingested_documents`	Documents extracted from emails or uploaded	`id`, `email_id`, `company_id`, `document_type`, `s3_key`, `extraction_status`, `extraction_result_json`, `confidence_score`, `created_at`
`migration_imports`	Batch import records from legacy systems	`id`, `company_id`, `source_system` (magaya, cargowise, descartes, csv, xml), `file_s3_key`, `record_count`, `success_count`, `error_count`, `status` (uploaded, parsing, validating, enriching, complete, failed), `created_at`, `completed_at`

Section 11 — Agent Architecture

The Cardinal Rule

> **Agents generate decisions. Deterministic services execute them.**

This rule is absolute and governs every interaction between the AI layer and the operational layer. An agent never writes to the database. An agent never calls an external API. An agent never sends an email. An agent never binds an insurance policy. An agent never creates a financial transaction. Agents produce structured JSON outputs — classifications, recommendations, extracted data, generated text, risk assessments, resolution proposals. Deterministic services consume those outputs and execute system actions after validation, business rules, and human approval.

This separation exists for three reasons. **Reliability:** LLM outputs are probabilistic and can hallucinate. Deterministic services are predictable and testable. **Auditability:** every agent output is logged with its reasoning, confidence score, and the model that produced it. Every service action is logged with the agent output that triggered it and the human approval that authorized it. **Reversibility:** if an agent produces an incorrect output, the damage is limited to a draft that an operator can correct. If an agent could write directly to the database, incorrect outputs would propagate through the system before anyone noticed.

Seven-Layer Execution Stack

Every operational action in Dynasties passes through seven layers in sequence. No layer can be skipped.

Layer 1 — Deterministic Workflow Engine. LangGraph manages the state machine for each workflow (email-to-shipment, import-from-legacy, exception-resolution, claim-preparation). The workflow engine determines which agents to invoke, in what order, and with what inputs. It handles branching (parallel processing of classification, compliance, and risk), merging (combining agent outputs into a shipment draft), and gating (holding execution until human approval is received).

Layer 2 — Rules Engine. Before any agent is invoked, the Rules Engine checks whether a deterministic rule already exists for the current input. If a commodity has been classified 3+ times with the same HS code and the operator has never corrected it, the Rules Engine assigns the HS code directly without invoking the Classification Agent. This reduces LLM costs, improves speed, and increases consistency. Rules are stored in the `rules` table and can be created manually by operators or learned automatically from operator corrections.

Layer 3 — Model Routing Service. When an agent needs to invoke an LLM, the request goes through the Model Routing Service. This service selects the appropriate model based on the task type, manages fallback chains (if the primary model is unavailable or returns

low-confidence output, the request is retried with an alternative model), caches responses for identical inputs, tracks costs per model per task type, and manages prompt versions.

Layer 4 — Agent Reasoning Layer. The specialist agent receives the input and the relevant Company Memory Graph context, and produces a structured JSON output. The output includes the agent's recommendation, confidence score, reasoning explanation, and any flags or warnings.

Layer 5 — Tool Execution Layer. If the agent's reasoning requires external data (a compliance database lookup, a carrier API call, an insurance quote request), the agent does not make the call directly. It produces a structured tool request, and the Tool Execution Layer makes the call and returns the result to the agent. This ensures that all external calls are logged, rate-limited, and error-handled consistently.

Layer 6 — Human Approval Checkpoint. The agent's output is presented to the operator in the Agentic Workbench. The operator can approve, modify, or reject. Modifications are captured as learning events. Approvals trigger Layer 7.

Layer 7 — System-of-Record Transaction Layer. Deterministic services execute the approved action: creating a database record, generating a document, binding an insurance policy, sending an invoice, or resolving an exception. All transactions are atomic and logged.

Specialist Agents

Agent	Primary Model	Fallback Model	Input	Output
Intake Agent	Claude Sonnet 4.6	GPT-5.4	Raw email MIME content + attachments	Structured intake record: document types identified, shipment intent detected, urgency classification
Document Extraction Agent	Claude Sonnet 4.6	Gemini 2.5 Pro	Individual document (PDF, image, spreadsheet)	Structured JSON with all extracted fields, each with confidence score
Entity Resolution Agent	GPT-5 Mini	GPT-5.4	Extracted party names/addresses + Company	Resolved entity IDs or new entity drafts

			Memory Graph entities	with match confidence
Classification Agent	GPT-5.4	Claude Sonnet 4.6	Commodity description + Company Memory Graph precedents	HS code, tariff classification, confidence score, precedent reference
Compliance Agent	GPT-5 Mini	GPT-5.4	Entity data + screening database results	Screening result (clear/potential match/confirmed match), match details, recommended action
Risk Intelligence Agent	GPT-5.4	Claude Sonnet 4.6	Shipment data + trade lane stats + carrier reliability + external risk data	Composite risk score (0-100), factor breakdown, mitigation recommendations
Cargo Insurance Agent	GPT-5.4	Claude Sonnet 4.6	Shipment data + risk score + cargo value + Company Memory Graph insurance history	Coverage recommendation, premium estimate, insurer selection, confidence score
Trade Lane Agent	GPT-5.4	Gemini 2.5 Pro	Trade lane identifier + historical data + current conditions	Cost prediction, transit time prediction, risk alerts, routing alternatives
Carrier Agent	GPT-5.4	GPT-5 Mini	Shipment parameters + Company Memory Graph carrier preferences + rate tables	Ranked carrier recommendations with cost, transit time, reliability, and reasoning

Pricing Agent	GPT-5 Mini	GPT-5.4	Shipment data + rate tables + historical pricing + Company Memory Graph patterns	Charge breakdown, margin analysis, anomaly flags
Document Generation Agent	Claude Sonnet 4.6	GPT-5.4	Approved shipment data + document template + Company Memory Graph preferences	Completed document fields ready for template rendering
Operations Exception Agent	GPT-5.4	Claude Sonnet 4.6	Exception data + shipment context + Company Memory Graph resolution history	Exception classification, impact assessment, proposed resolution, required communication s
Claims Preparation Agent	Claude Sonnet 4.6	GPT-5.4	Shipment data + policy data + damage report + supporting documents	Structured claim package: narrative, document manifest, loss calculation
Memory Agent	GPT-5 Mini	GPT-5.4	Operator correction event + Company Memory Graph context	Graph update instructions, rule proposals, pattern recognition results
Natural Language Agent	GPT-5.4	Claude Sonnet 4.6	Operator natural language input	Intent classification, structured parameters, workflow routing

Model Routing Strategy

Model	Primary Use Cases	Cost Tier	Context Window	Strengths
GPT-5.4	Complex reasoning, agentic orchestration, risk assessment, carrier analysis, trade lane prediction	High	128K tokens	Strongest reasoning, best for multi-factor analysis
Claude Sonnet 4.6	Document understanding, text extraction, document generation, claim narratives	High	200K tokens	Superior document comprehension, excellent structured output
Gemini 2.5 Pro	Large-context analysis, bulk document processing, historical data analysis	Medium	1M tokens	Massive context window for processing entire document sets
GPT-5 Mini	High-volume operational tasks: entity resolution, compliance screening, pricing calculation, memory updates	Low (10x cheaper)	128K tokens	Fast, cost-effective for structured, well-defined tasks

The Model Routing Service implements three cost optimization strategies. **Caching:** identical inputs (same document, same entity, same commodity description) return cached results without invoking the LLM. **Cascading:** simple tasks start with GPT-5 Mini; if the confidence score is below threshold, the task is escalated to GPT-5.4 or Claude Sonnet 4.6. **Rules Engine bypass:** when a deterministic rule exists, the LLM is not invoked at all.

Section 12 — The Core System Rule

Agents Generate Decisions. Deterministic Services Execute Them.

This section exists to reinforce the single most important architectural principle in the entire platform. Every engineer, every code review, and every architectural decision must be evaluated against this rule. If a proposed change allows an agent to write to the database, call an external API, send a communication, or modify a financial record, the change is rejected. There are no exceptions.

Why this rule exists:

Language models are probabilistic systems. They produce outputs that are usually correct but occasionally wrong. In a consumer application, an occasional error is an inconvenience. In a freight forwarding operation, an incorrect HS code can trigger a customs seizure. An incorrect party screening can result in a sanctions violation. An incorrect charge calculation can destroy a client relationship. An incorrect insurance coverage recommendation can leave cargo uninsured during a total loss event.

The seven-layer execution stack ensures that every agent output is validated before it affects the real world. The Rules Engine catches known patterns. The confidence scoring flags uncertain outputs. The human approval checkpoint provides the final gate. The deterministic service layer executes only approved actions with full transaction logging.

Implementation enforcement:

The Agent Worker Service runs in an isolated compute environment with no direct database write access. It can read from the database (via read-only connection) and from the Company Memory Graph. It publishes structured JSON outputs to SQS queues. The API Gateway Service, Shipment Construction Service, Insurance Service, Billing Service, and other execution services consume those outputs and perform the writes. The network architecture physically prevents agents from bypassing the execution layer.

Section 13 — Replit Agent 4 Design System

Design System Architecture

The Dynasties UI is built using React, Next.js, TailwindCSS, and the shadcn/ui component system. The design system is optimized for Replit Agent 4 and Replit Design Mode, enabling rapid design iteration on an infinite canvas with visual variant exploration and theme-driven component consistency.

Design Tokens

All visual properties are defined as CSS custom properties (design tokens) in a centralized theme file. Components never use hardcoded color values, font sizes, or spacing values. They reference tokens exclusively. This ensures that a single token change propagates through every component in the application.

Color Tokens:

Token	Purpose	Light Mode Value	Dark Mode Value
`--background`	Page background	`oklch(0.98 0.005 250)`	`oklch(0.14 0.02 260)`
`--foreground`	Primary text	`oklch(0.15 0.02 260)`	`oklch(0.95 0.005 250)`
`--card`	Card/panel background	`oklch(0.99 0.003 250)`	`oklch(0.18 0.02 260)`
`--card-foreground`	Card text	`oklch(0.15 0.02 260)`	`oklch(0.95 0.005 250)`
`--primary`	Primary actions, active states	`oklch(0.55 0.18 250)`	`oklch(0.65 0.18 250)`
`--primary-foreground`	Text on primary backgrounds	`oklch(0.98 0.005 250)`	`oklch(0.12 0.02 260)`
`--secondary`	Secondary actions, subtle backgrounds	`oklch(0.92 0.01 250)`	`oklch(0.22 0.02 260)`
`--accent`	Highlights, badges, emphasis	`oklch(0.70 0.15 160)`	`oklch(0.70 0.15 160)`
`--destructive`	Error states, critical alerts	`oklch(0.55 0.22 25)`	`oklch(0.65 0.22 25)`
`--muted`	Disabled states, placeholder text	`oklch(0.92 0.005 250)`	`oklch(0.25 0.01 260)`
`--border`	Borders, dividers	`oklch(0.88 0.01 250)`	`oklch(0.28 0.02 260)`
`--ring`	Focus rings	`oklch(0.55 0.18 250)`	`oklch(0.65 0.18 250)`

Semantic Color Tokens for Operational States:

Token	Purpose	Value

`--status-approved`	Approved shipments, passed compliance	`oklch(0.65 0.18 150)`
`--status-pending`	Pending review, awaiting approval	`oklch(0.75 0.15 80)`
`--status-flagged`	Compliance flags, risk warnings	`oklch(0.65 0.20 40)`
`--status-critical`	Critical exceptions, sanctions matches	`oklch(0.55 0.22 25)`
`--confidence-high`	Agent confidence > 90%	`oklch(0.65 0.18 150)`
`--confidence-medium`	Agent confidence 70-90%	`oklch(0.75 0.15 80)`
`--confidence-low`	Agent confidence < 70%	`oklch(0.65 0.20 40)`

Typography Tokens:

Token	Purpose	Value
`--font-display`	Headings, page titles	`"Space Grotesk", system-ui, sans-serif`
`--font-body`	Body text, descriptions	`"Inter", system-ui, sans-serif`
`--font-mono`	Reference numbers, codes, data	`"JetBrains Mono", "Fira Code", monospace`
`--text-xs`	Metadata, timestamps	`0.75rem / 1rem`
`--text-sm`	Secondary text, labels	`0.875rem / 1.25rem`
`--text-base`	Body text	`1rem / 1.5rem`
`--text-lg`	Section headings	`1.125rem / 1.75rem`
`--text-xl`	Page titles	`1.25rem / 1.75rem`
`--text-2xl`	Dashboard metrics	`1.5rem / 2rem`
`--text-3xl`	Hero numbers	`1.875rem / 2.25rem`

Spacing Tokens:

The spacing system uses a 4px base unit with a consistent scale: `--space-1` (4px), `--space-2` (8px), `--space-3` (12px), `--space-4` (16px), `--space-5` (20px), `--space-6` (24px), `--space-8` (32px), `--space-10` (40px), `--space-12` (48px), `--space-16` (64px).

Component Architecture

All components extend shadcn/ui primitives. Custom components are built by composing shadcn/ui base components with Dynasties-specific variants and operational semantics.

Core Operational Components:

Component	Base	Purpose
`ShipmentCard`	`Card`	Displays shipment summary in queue views with status badge, risk indicator, and confidence score
`ApprovalPanel`	`Sheet`	Side panel showing full shipment draft with all agent outputs, confidence scores, and approval controls
`ConfidenceIndicator`	`Badge`	Visual indicator of agent confidence level (high/medium/low) with color coding
`RiskBadge`	`Badge`	Risk score display with color gradient from green (low) to red (critical)
`ComplianceResult`	`Alert`	Compliance screening result with match details and recommended action
`ChargeTable`	`Table`	Charge breakdown with rate source indicators and anomaly flags
`InsuranceQuoteCard`	`Card`	Insurance quote display with coverage details, premium, and one-click bind
`ExceptionAlert`	`Alert`	Exception notification with severity, impact assessment, and resolution actions

`ClaimTimeline`	Custom	Vertical timeline showing claim lifecycle with status transitions and communications
`OperationalQueue`	Custom	Filterable, sortable list of items (shipments, exceptions, claims) with batch actions
`AgentReasoningPanel`	`Collapsible`	Expandable panel showing agent reasoning, model used, and confidence breakdown
`EntityCard`	`Card`	Entity display with name, address, type, and relationship indicators
`TradeLineChart`	Custom	Trade lane intelligence visualization with cost/time predictions and confidence intervals
`MemoryGraphView`	Custom	Visual representation of Company Memory Graph relationships for a shipment

Replit Design Mode Integration

The design system is structured so that Replit Agent 4 can generate, modify, and iterate on UI components using the infinite canvas and Design Mode workflow.

Theme File Structure: All design tokens are defined in a single `globals.css` file that Replit Agent 4 can modify to explore different visual directions. Changing a token value automatically updates every component that references it, enabling rapid theme exploration on the canvas.

Component Isolation: Each component is self-contained with its own file, props interface, and variant definitions. This allows Replit Agent 4 to generate multiple variants of a component on the canvas, compare them side by side, and apply the best version to the production codebase.

Responsive Previews: Components are built with responsive breakpoints that align with Replit Design Mode's device preview sizes (mobile 390x844, tablet 768x1024, desktop 1280x720). Operators in the field may use tablets; the interface must be fully functional at tablet resolution.

Design-to-Code Workflow: The recommended workflow for UI development is: (1) describe the desired interface to Replit Agent 4, (2) generate multiple variants on the canvas, (3) refine the best variant using visual controls and annotations, (4) apply the design to the production codebase, (5) verify that the generated code uses the correct design tokens and shadcn/ui primitives. If the generated code uses hardcoded values instead of tokens, correct it before merging.

Section 14 — Engineering Build Roadmap

Team Structure

The Phase One build assumes a two-person engineering team assisted by Replit Agent 4 for code generation, with the founder handling product decisions, pilot customer relationships, and insurance partner negotiations.

Role	Responsibilities
Full-Stack Engineer	API Gateway, Shipment Construction, Entity Resolution, Document Generation, Billing/Invoicing, Pricing, frontend (Agentic Workbench), authentication, deployment
AI/ML Engineer	Agent Worker Service, Model Routing Service, Workflow Orchestrator, all specialist agents, Company Memory Graph, Rules Engine, Email Ingestion, Document Extraction
Founder	Product direction, pilot customer onboarding, insurance partner selection and contract, compliance data source licensing, design reviews

Week 1: Foundation (Days 1-7)

Day	Deliverable	Owner
1	Project initialization in Replit: Next.js + TailwindCSS + shadcn/ui. Design system tokens in `globals.css`. PostgreSQL database with pgvector extension. Prisma	Full-Stack

	ORM schema for all tables in Section 10. Initial migration.	
1-2	AWS infrastructure: SES inbound email configuration, S3 buckets (raw-emails, documents, generated-documents), SQS queues (all 10 task queues + dead-letter queues), SNS topics (all 10 event topics). Terraform or CDK definitions.	AI/ML
2-3	Email Ingestion Service: SES → Lambda → S3 → SQS pipeline. MIME parsing, attachment extraction, document type classification. End-to-end test: send email, verify parsed output in queue.	AI/ML
3-4	Document Extraction pipeline: Claude Sonnet 4.6 integration via Model Routing Service. Extraction prompts for BOL, commercial invoice, packing list. Structured JSON output with confidence scores.	AI/ML
3-4	API Gateway Service: Express.js with authentication middleware (JWT). REST endpoints for shipments, entities, documents, charges, invoices. Request validation. CORS configuration.	Full-Stack
4-5	Shipment Construction Service: create shipment from structured extraction output. All fields from `shipments` table. Status lifecycle management. Container management.	Full-Stack
5-6	Entity Resolution Service: fuzzy matching against	Full-Stack

	existing entities (Levenshtein distance + embedding similarity via pgvector). Address normalization. Auto-creation of new entity drafts.	
6-7	Migration Studio backend: Magaya CSV parser using XSD-derived field mappings (243 fields mapped). Magaya XML parser. CargoWise common field mappings. AI-assisted mapping for unknown CSV formats. Batch import with row-level validation.	AI/ML
7	Workflow Orchestrator: LangGraph workflow definitions for "New Shipment from Email" and "Import from Legacy System." State management, checkpointing, parallel branches for classification + compliance + risk.	AI/ML

Week 1 Milestone: Forward an email with a commercial invoice attachment → see a structured shipment draft with extracted data and confidence scores. Upload a Magaya CSV export → see imported shipment records with validation results.

Week 2: Intelligence Layer (Days 8-14)

Day	Deliverable	Owner
8-9	Classification Agent: HS code suggestion with tariff schedule context. Confidence scoring. Company Memory Graph precedent lookup. Rules Engine integration for known commodities.	AI/ML
8-9	Compliance Screening Service: restricted party screening against SDN, Entity List, Denied Persons, UN/EU	AI/ML

	sanctions lists. Mandatory, non-bypassable. Screening result storage with match details for audit trail.	
9-10	Risk Intelligence Service: composite risk score calculation from trade lane risk, carrier reliability, commodity risk, and historical delay data. Factor weighting. Risk Intelligence Agent for mitigation recommendations.	AI/ML
10-11	Rules Engine: YAML-based business rules, evaluation pipeline, deterministic handling for known patterns. Initial rule set for common commodities and compliance scenarios.	AI/ML
10-11	Company Memory Graph: PostgreSQL + pgvector schema for nodes and edges. Query API for agents. Initial graph population from imported data.	AI/ML
11-12	Insurance Service: Loadsure or Breeze API integration (adapter pattern). Quote generation from shipment parameters + risk score. Cargo Insurance Agent for coverage recommendation.	AI/ML
12-13	Agentic Workbench frontend: React + TailwindCSS + shadcn/ui. Seven operational queues (Drafts, Missing Info, Compliance Flags, Risk Alerts, Insurance Pending, Approvals, Exceptions). Shipment detail view with confidence indicators. Keyboard shortcuts. Approval workflow.	Full-Stack

13-14	Document Generation Agent + Service: BOL, AWB, Cargo Manifest, Arrival Notice, Certificate of Insurance template engine. Claude Sonnet 4.6 for intelligent field filling. PDF generation. S3 storage.	Full-Stack
14	End-to-end integration: email → intake → extraction → entity resolution → classification + compliance + risk (parallel) → insurance quote → shipment draft → Workbench → approval → document generation.	Both

Week 2 Milestone: Complete email-to-shipment-with-documents-and-insurance flow in the Agentic Workbench. Operator sees drafts with HS codes, compliance results, risk scores, insurance quotes, confidence indicators, and generated documents. Approve with one click.

Week 3: Financial Loop, Exceptions, and Claims (Days 15-21)

Day	Deliverable	Owner
15-16	Pricing Agent + Service: charge type management, rate table CRUD, auto-calculation from shipment parameters, margin analysis, anomaly detection against historical patterns.	Full-Stack
16-17	Billing and Invoicing Service: invoice generation from approved charges, PDF output, email delivery via SES, accounts receivable tracking with aging, payment recording.	Full-Stack
17-18	Trade Lane Intelligence Service: aggregate statistics from historical shipment data. Trade Lane Agent for	AI/ML

	cost prediction, transit time prediction, and risk alerting.	
18-19	Exception Management Service: exception detection rules (missing documents, customs hold detection, delay detection, billing discrepancy detection). Operations Exception Agent for resolution proposals. Exception queue in Workbench.	AI/ML
19-20	Claims Management Service: incident recording, evidence collection workflow, Claims Preparation Agent for automated claim package assembly, insurer communication logging, claim status tracking. Claims view in Workbench.	Both
20-21	Natural Language Interface: GPT-5.4 intent parsing, routing to workflows, structured responses. "Create a shipment from Shanghai to LA, electronics, 2 TEU" → full shipment draft. Carrier Agent for ranked carrier recommendations.	AI/ML
21	Memory Agent: captures operator corrections, updates Company Memory Graph, generates rule proposals from repeated patterns, tracks learning metrics (auto-approval rate).	AI/ML

Week 3 Milestone: Complete financial loop closed — shipment → documents → charges → invoice → email delivery. Exception detection and resolution workflow operational. Claims preparation workflow operational. Trade lane predictions available. Natural language shipment creation works. Memory Agent learning from every operator correction.

Week 4: Security, Migration Studio, Polish, and Launch (Days 22-30)

Day	Deliverable	Owner
22-23	Authentication: OAuth 2.0, JWT, role-based access (Admin, Manager, Operator, Viewer). Session management. Password reset.	Full-Stack
23-24	Migration Studio frontend: guided import wizard with file upload, source system selection (Magaya, CargoWise, Descartes, CSV, XML), AI-assisted field mapping review, validation summary with row-level error details, enrichment results, import confirmation. Free Intelligence Audit report generation (PDF).	Full-Stack
24-25	Onboarding flow: account creation, email forwarding setup instructions (with DNS verification), guided first-shipment experience, Migration Studio walkthrough.	Full-Stack
25-26	Security hardening: input validation on all endpoints, rate limiting, CORS configuration, SQL injection prevention (Prisma parameterized queries), XSS protection, encryption at rest (AWS KMS) and in transit (TLS 1.3).	Full-Stack
26-27	Workbench polish: batch operations (approve multiple shipments, bulk exception resolution), advanced filtering and sorting, responsive design for tablet, loading states, error handling,	Full-Stack

	empty states, toast notifications.	
27-28	End-to-end testing with real shipment data from pilot customer. Bug fixes, performance optimization, edge case handling across all workflows (email-to-shipment, migration, exception, claims).	Both
28-29	Production deployment: AWS ECS Fargate services, RDS PostgreSQL, ElastiCache Redis, CloudFront CDN, Route 53 DNS (`app.dynasties.ai`), SSL certificates, CloudWatch alarms, error tracking (Sentry). Blue-green deployment pipeline.	Full-Stack
29-30	Launch with pilot customer. White-glove onboarding: founder walks customer through account setup, email forwarding configuration, Magaya data import via Migration Studio, and first live shipment through the complete workflow.	Founder

Day 30 Milestone: Dynasties is live. First pilot customer is processing real shipments through the Agentic Workbench with full intelligence (classification, compliance, risk, insurance, trade lane predictions), exception handling, and claims management. The Company Memory Graph is learning. The complete operational loop from email ingestion through invoicing is functional. Monitoring and alerting are active.

Post-Launch: Weeks 5-8

Week	Priority	Rationale
5-6	QuickBooks / Xero invoice sync	Eliminates manual accounting entry — top customer request

5-6	Carrier tracking API integrations (Maersk, MSC, CMA CGM)	Automates milestone updates, enables ETA predictions, feeds Exception Management
6-7	Customer-facing tracking portal	Differentiation, reduces "where is my shipment" calls by 80%
6-7	Additional insurer integrations (second and third provider)	Better coverage options, competitive quoting, higher attachment rates
7-8	Advanced analytics dashboard	Retention driver — operational intelligence visualized as strategic advisor
7-8	Second and third pilot customers	Validates platform across different operational patterns, different trade lanes, different company sizes

---

Section 15 — Output Specification

What This Document Produces

This framework document is designed to serve as the complete input specification for building Dynasties Phase One. When ingested by Replit Agent 4 (or used by a human engineering team), it should produce the following outputs.

Application Codebase:

A Next.js application with React frontend and Node.js backend services, using TailwindCSS and shadcn/ui for the design system, Prisma ORM for database access, LangGraph for workflow orchestration, and the Model Routing Service for all LLM interactions. The codebase implements all services described in Section 2, all agents described in Section 11, all database tables described in Section 10, and all UI components described in Section 13.

Infrastructure Configuration:

Terraform or AWS CDK definitions for all AWS resources: ECS Fargate services, RDS PostgreSQL with pgvector, ElastiCache Redis, S3 buckets, SQS queues, SNS topics, SES inbound email configuration, CloudFront CDN, Route 53 DNS, KMS encryption keys, IAM roles and policies, CloudWatch alarms, and VPC networking.

Design System:

A complete design token file (`globals.css`) with all color, typography, and spacing tokens. A component library extending shadcn/ui with all operational components listed in Section 13. Responsive layouts tested at mobile, tablet, and desktop breakpoints.

Operational Workflows:

LangGraph workflow definitions for: email-to-shipment, legacy-system-import, exception-detection-and-resolution, claim-preparation-and-submission, and natural-language-command-execution. Each workflow implements the seven-layer execution stack described in Section 11.

Agent Prompts:

Versioned prompt templates for all 15 specialist agents, stored in the Model Routing Service with A/B testing capability. Each prompt includes the agent's role definition, input schema, output schema, Company Memory Graph query instructions, and confidence scoring criteria.

Success Criteria

Phase One is complete when the following conditions are met:

The platform can receive an email with a booking request and commercial invoice, extract all shipment data, resolve entities, classify commodities, screen for compliance, calculate a risk score, generate an insurance quote, predict trade lane costs and timing, calculate charges, and present a complete shipment draft in the Agentic Workbench — all within 30 seconds of email receipt.

An operator can approve the shipment with one click, triggering automatic document generation (BOL/AWB), insurance policy binding, invoice generation, and email delivery — all within 5 seconds of approval.

The Migration Studio can import a Magaya CSV export of 1,000+ shipments, validate all records, enrich with intelligence, and present results — within 10 minutes.

The Exception Management Service detects missing documents, potential customs holds, and billing discrepancies automatically, and proposes resolutions that operators can execute with one click.

The Claims Management Service can assemble a complete insurance claim package from existing shipment data and operator-provided damage reports — within 5 minutes.

The Company Memory Graph learns from every operator correction and improves agent accuracy over time, with a measurable increase in auto-approval rate week over week.

References

[1] What Is Blue Ocean Strategy? — Blue Ocean Strategy
[2] Zero to One — Peter Thiel
[3] Transportation Management System Market Size — Grand View Research
[4] WiseTech Global Investor Relations
[5] Descartes Systems Group Investor Relations
[6] e2open Investor Relations
[7] Introducing Replit Agent 4: Built for Creativity — Replit Blog
[8] Design Mode — Replit Documentation
[9] Magaya Supply Chain — Working with Lists
[10] OpenAI GPT-5 Model Guide — NxCode
[11] Gemini 2.5 Pro Specs and Pricing — GetDeploying
[12] Loadsure — Instant Digital Cargo Insurance
[13] Breeze AI — Embedded Cargo Insurance
[14] Best Multi-Agent Frameworks in 2026 — GurusUp
[15] Agent 4 — The Next Evolution in AI App Building Tools — Replit